

# PROGRAMACIÓN IV — UTN TUP A Distancia

## Guía Paso a Paso: Primer Parcial

*Aplicación Fullstack con FastAPI + React*

*Esta guía fue pensada para que construyas el proyecto entendiendo cada decisión técnica, no solo copiando código. Cada sección explica el QUÉ, el CÓMO y el POR QUÉ.*

## Introducción: Cómo pensar este proyecto

Antes de escribir una sola línea de código, es fundamental entender qué estás construyendo. Este proyecto es una **aplicación fullstack**: tiene una parte de servidor (el **backend**) y una parte visual que usa el usuario (el **frontend**). Ambas partes se comunican entre sí a través de una API HTTP.

El dominio del sistema es una **tienda o catálogo de productos**. Pensá en algo como una app de recetas o un menú de restaurante: tenés **categorías** (ej. Lácteos, Panificados), **productos** que pertenecen a esas categorías, e **ingredientes** que componen esos productos. El UML que te dieron describe exactamente esa estructura.

La arquitectura que vas a usar se puede resumir así:

```
Usuario → React (Frontend)
          ↓ HTTP/JSON
        FastAPI (Backend)
          ↓ SQL
    PostgreSQL (Base de datos)
```

Cada capa tiene una responsabilidad clara: React **muestra**, FastAPI **procesa**, y PostgreSQL **persiste**. Entender eso te va a ayudar a saber dónde poner cada cosa.

## PARTE 1 — El Backend con FastAPI y SQLAlchemy

El backend es el **corazón lógico de la aplicación**. No tiene interfaz visual; su trabajo es recibir pedidos HTTP, ejecutar lógica de negocio, y devolver respuestas en formato JSON. Usaremos **FastAPI** como framework y **SQLModel** para el manejo de la base de datos.

### 1.1 — Estructura de carpetas del backend

Organizar bien el código desde el principio es tan importante como escribirlo. Una buena estructura hace que el proyecto sea fácil de mantener y de explicar en el video. La propuesta es la siguiente:

```

backend/
├── app/
│   ├── main.py                # Punto de entrada: crea la app FastAPI
│   ├── database.py            # Conexión a PostgreSQL
│   ├── models/                # Clases SQLAlchemy (tablas de la DB)
│   │   ├── categoria.py
│   │   ├── producto.py
│   │   ├── ingrediente.py
│   │   ├── producto_categoria.py
│   │   └── producto_ingrediente.py
│   ├── schemas/               # Pydantic: lo que entra y sale de la API
│   │   ├── categoria.py
│   │   ├── producto.py
│   │   └── ingrediente.py
│   ├── routers/               # Endpoints agrupados por recurso
│   │   ├── categorias.py
│   │   ├── productos.py
│   │   └── ingredientes.py
│   ├── services/              # Lógica de negocio (operaciones DB)
│   │   ├── categoria_service.py
│   │   ├── producto_service.py
│   │   └── ingrediente_service.py
│   └── uow.py                  # Unit of Work: gestión de sesión/transacción
├── requirements.txt
└── .env                        # Variables de entorno (connection string)

```

 ¿Por qué separar `models/` de `schemas/`? Porque el modelo (SQLModel) representa la tabla en la base de datos, mientras que el schema (Pydantic) representa lo que querés exponer en la API. Son cosas distintas: la tabla puede tener campos internos (como `deleted_at`) que no querés enviar al frontend.

## 1.2 — Entorno virtual y dependencias

Un **entorno virtual** (`.venv`) es una carpeta aislada que contiene las librerías Python específicas de este proyecto. Evita conflictos con otras versiones instaladas en tu computadora. Es obligatorio para cualquier proyecto Python serio.

Para crearlo y activarlo ejecutá en la terminal desde la carpeta `backend/`:

```

# Crear el entorno virtual
python -m venv .venv

# Activarlo (Windows PowerShell)
.venv\Scripts\Activate.ps1

# Activarlo (Linux/macOS)
source .venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt

```

Tu `requirements.txt` debe contener al menos:

```

fastapi
uvicorn[standard]
sqlmodel
psycopg2-binary

```

```
python-dotenv
alembic          # opcional, para migraciones
```

 *uvicorn es el servidor web que ejecuta FastAPI. psycopg2-binary es el driver que conecta Python con PostgreSQL. python-dotenv permite leer variables de entorno desde un archivo .env.*

### 1.3 — Conexión a PostgreSQL (database.py)

La conexión a la base de datos se configura en `database.py`. `SQLModel` usa **SQLAlchemy** internamente, por lo que vas a trabajar con el concepto de **engine** (motor de base de datos) y **session** (una conexión activa con transacción).

```
# app/database.py
from sqlmodel import create_engine, Session, SQLModel
from dotenv import load_dotenv
import os

load_dotenv() # Carga las variables del archivo .env

DATABASE_URL = os.getenv('DATABASE_URL')
# Ejemplo de DATABASE_URL en .env:
# DATABASE_URL=postgresql://usuario:password@localhost:5432/mi_db

engine = create_engine(DATABASE_URL, echo=True)
# echo=True imprime las queries SQL en consola. Muy útil para desarrollo.

def create_db_and_tables():
    # Crea las tablas que todavía no existen en la DB
    SQLModel.metadata.create_all(engine)

def get_session():
    # Generador que FastAPI usa como dependencia inyectable
    with Session(engine) as session:
        yield session # 'yield' en lugar de 'return': mantiene la sesión abierta
```

 *El patrón 'yield' en `get_session()` es clave: FastAPI lo usa como dependencia. La sesión se abre antes de ejecutar el endpoint, y se cierra (con `commit` o `rollback`) automáticamente después. Esto evita que se quede una conexión abierta a la DB si hay un error.*

### 1.4 — Modelado de datos con SQLModel

`SQLModel` combina **Pydantic** (validación) y **SQLAlchemy** (ORM) en una sola clase. Con `table=True` le decís que esa clase representa una tabla real en la base de datos. Sin ese parámetro, es solo un schema de validación Pydantic.

Empecemos con el modelo más simple, **Ingrediente**, que no tiene relaciones hacia arriba:

```
# app/models/ingrediente.py
from sqlmodel import SQLModel, Field, Relationship
from typing import Optional
from datetime import datetime
```

```

class Ingrediente(SQLModel, table=True):
    # PK: BIGSERIAL equivale a int con auto-increment en PostgreSQL
    id: Optional[int] = Field(default=None, primary_key=True)

    # UQ, NN: unique=True y no puede ser None
    nombre: str = Field(max_length=100, unique=True)
    descripcion: Optional[str] = None

    # es_alergeno tiene DEFAULT false en la DB
    es_alergeno: bool = Field(default=False)

    # Campos de auditoria (se setean automáticamente)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)

    # Relación inversa: este ingrediente puede estar en muchos ProductoIngrediente
    producto_links: list['ProductoIngrediente'] =
Relationship(back_populates='ingrediente')

```

Ahora el modelo Producto, que es el más complejo porque tiene varias relaciones:

```

# app/models/producto.py
from sqlmodel import SQLModel, Field, Relationship
from typing import Optional
from datetime import datetime

class Producto(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)

    # FK hacia UnidadMedida (puede ser NULL según el UML)
    unidad_venta_id: Optional[int] = Field(default=None,
foreign_key='unidadmedida.id')

    nombre: str = Field(max_length=150)
    descripcion: Optional[str] = None
    precio_base: float = Field(ge=0) # ge=0 → CHECK >= 0
    imagenes_url: Optional[str] = None
    stock_cantidad: int = Field(default=0, ge=0)
    disponible: bool = Field(default=True)

    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)
    deleted_at: Optional[datetime] = None # Soft delete

    # Relaciones N:N (a través de tablas intermedias)
    categoria_links: list['ProductoCategoria'] =
Relationship(back_populates='producto')
    ingrediente_links: list['ProductoIngrediente'] =
Relationship(back_populates='producto')

```

La tabla intermedia ProductoIngrediente es la que implementa la relación N:N entre Producto e Ingrediente. Es el patrón más importante del parcial:

```

# app/models/producto_ingrediente.py
from sqlmodel import SQLModel, Field, Relationship
from typing import Optional

class ProductoIngrediente(SQLModel, table=True):

```

```

# PK compuesta: ambos campos son FK y PK al mismo tiempo
producto_id: int = Field(foreign_key='producto.id', primary_key=True)
ingrediente_id: int = Field(foreign_key='ingrediente.id', primary_key=True)

# Atributos propios de la relación (según el UML)
cantidad: float = Field(gt=0) # gt=0 → CHECK > 0
unidad_medida_id: int = Field(foreign_key='unidadmedida.id')
es_removible: bool = Field(default=True)

# Relaciones hacia las entidades principales
producto: Optional['Producto'] =
Relationship(back_populates='ingrediente_links')
ingrediente: Optional['Ingrediente'] =
Relationship(back_populates='producto_links')

```

 *back\_populates crea la conexión bidireccional entre los modelos. Si accedés a `producto.ingrediente_links`, `SQLModel` automáticamente te trae todos los registros de `ProductoIngrediente` asociados a ese producto. Sin `back_populates`, la relación es de un solo lado.*

 **Acordate de importar todos los modelos antes de llamar a `create_all()`. `SQLModel` necesita conocer todas las clases para resolver las referencias de FK. Si no, podés recibir errores de 'tabla no encontrada'.**

## 1.5 — Schemas de entrada y salida (schemas/)

Los schemas sirven para **separar la representación interna (DB) de la representación pública (API)**. Esto se llama el patrón **DTO (Data Transfer Object)**. Para cada entidad vas a necesitar al menos tres schemas: uno para crear, uno para actualizar, y uno para leer.

```

# app/schemas/ingrediente.py
from pydantic import BaseModel
from typing import Optional

# Lo que el cliente manda cuando CREA un ingrediente
class IngredienteCreate(BaseModel):
    nombre: str
    descripcion: Optional[str] = None
    es_alergeno: bool = False

# Lo que el cliente manda cuando EDITA un ingrediente (todo opcional)
class IngredienteUpdate(BaseModel):
    nombre: Optional[str] = None
    descripcion: Optional[str] = None
    es_alergeno: Optional[bool] = None

# Lo que la API DEVUELVE (incluye id y campos de auditoría)
class IngredienteRead(BaseModel):
    id: int
    nombre: str
    descripcion: Optional[str]
    es_alergeno: bool

class Config:
    from_attributes = True # Permite crear este schema desde un objeto ORM

```

 La Config con `from_attributes = True` (antes llamado `orm_mode`) le dice a Pydantic que puede construir el schema leyendo atributos de un objeto Python (como una instancia de `Ingrediente` del ORM), no solo desde un diccionario. Sin esto, `IngredienteRead(*objeto_orm*)` fallaría.

## 1.6 — Routers y endpoints (routers/)

Los routers agrupan todos los endpoints de un recurso. FastAPI usa `APIRouter` para esto. El parcial exige que uses **Annotated** con **Query** para filtros o paginación. Veamos un ejemplo completo para `Ingrediente`:

```
# app/routers/ingredientes.py
from fastapi import APIRouter, Depends, HTTPException, Query
from typing import Annotated
from sqlmodel import Session
from ..database import get_session
from ..schemas.ingrediente import IngredienteCreate, IngredienteRead,
IngredienteUpdate
from ..services import ingrediente_service

router = APIRouter(prefix='/ingredientes', tags=['Ingredientes'])

# Annotated combina el tipo con la validación del Query en una sola línea
# Es más limpio que poner todo en la firma de la función
@router.get('/', response_model=list[IngredienteRead])
def listar_ingredientes(
    session: Session = Depends(get_session),
    skip: Annotated[int, Query(ge=0, description='Registros a omitir')] = 0,
    limit: Annotated[int, Query(ge=1, le=100, description='Máx registros')] = 20,
    es_alergeno: Annotated[bool | None, Query()] = None,
):
    return ingrediente_service.get_all(session, skip, limit, es_alergeno)

@router.post('/', response_model=IngredienteRead, status_code=201)
def crear_ingrediente(
    data: IngredienteCreate,
    session: Session = Depends(get_session)
):
    return ingrediente_service.create(session, data)

@router.get('/{ingrediente_id}', response_model=IngredienteRead)
def obtener_ingrediente(
    ingrediente_id: int,
    session: Session = Depends(get_session)
):
    item = ingrediente_service.get_by_id(session, ingrediente_id)
    if not item:
        raise HTTPException(status_code=404, detail='Ingrediente no encontrado')
    return item

@router.patch('/{ingrediente_id}', response_model=IngredienteRead)
def actualizar_ingrediente(
    ingrediente_id: int,
    data: IngredienteUpdate,
    session: Session = Depends(get_session)
):
    item = ingrediente_service.update(session, ingrediente_id, data)
    if not item:
```

```

        raise HTTPException(status_code=404, detail='Ingrediente no encontrado')
    return item

@router.delete('/{ingrediente_id}', status_code=204)
def eliminar_ingrediente(
    ingrediente_id: int,
    session: Session = Depends(get_session)
):
    ok = ingrediente_service.delete(session, ingrediente_id)
    if not ok:
        raise HTTPException(status_code=404, detail='Ingrediente no encontrado')

```

 *Usá PATCH (actualización parcial) en lugar de PUT (reemplaza todo el objeto). Así el cliente puede mandar solo los campos que quiere cambiar, que es lo que IngredienteUpdate permite con todos sus campos Optional.*

## 1.7 — Capa de servicios (services/)

Los servicios contienen la **lógica de negocio real**: las consultas a la DB, las validaciones, las transformaciones. Los routers no deberían hacer nada con la DB directamente; solo delegan al servicio y manejan los códigos HTTP. Esto hace al código más testeable y más limpio.

```

# app/services/ingrediente_service.py
from sqlmodel import Session, select
from ..models.ingrediente import Ingrediente
from ..schemas.ingrediente import IngredienteCreate, IngredienteUpdate
from typing import Optional

def get_all(session: Session, skip: int, limit: int,
            es_alergeno: Optional[bool]) -> list[Ingrediente]:
    query = select(Ingrediente)
    if es_alergeno is not None:
        query = query.where(Ingrediente.es_alergeno == es_alergeno)
    return session.exec(query.offset(skip).limit(limit)).all()

def get_by_id(session: Session, ingrediente_id: int) -> Optional[Ingrediente]:
    return session.get(Ingrediente, ingrediente_id)

def create(session: Session, data: IngredienteCreate) -> Ingrediente:
    # Convertimos el schema Pydantic a un objeto del modelo ORM
    item = Ingrediente.model_validate(data)
    session.add(item)
    session.commit()
    session.refresh(item) # Recarga el objeto con el ID generado por la DB
    return item

def update(session: Session, ingrediente_id: int,
           data: IngredienteUpdate) -> Optional[Ingrediente]:
    item = session.get(Ingrediente, ingrediente_id)
    if not item:
        return None
    # model_dump(exclude_unset=True): solo los campos que el cliente mandó
    update_data = data.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(item, key, value)
    session.add(item)
    session.commit()

```

```

    session.refresh(item)
    return item

def delete(session: Session, ingrediente_id: int) -> bool:
    item = session.get(Ingrediente, ingrediente_id)
    if not item:
        return False
    session.delete(item)
    session.commit()
    return True

```

 *exclude\_unset=True en model\_dump() es fundamental para el PATCH. Si el cliente manda solo {nombre: 'Sal'}, exclude\_unset filtra el resto de campos (descripcion, es\_alergeno) y no los pisa con None en la DB.*

## 1.8 — Punto de entrada (main.py)

En `main.py` creás la aplicación FastAPI, configurás CORS (necesario para que React pueda hacer requests al backend), registrás los routers y creás las tablas al iniciar:

```

# app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from .database import create_db_and_tables
from .routers import categorias, productos, ingredientes

app = FastAPI(title='API Parcial 1', version='1.0.0')

# CORS: permite que el frontend (en otro puerto) haga requests a esta API
app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:5173'], # Puerto de Vite
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

@app.on_event('startup')
def on_startup():
    create_db_and_tables()

app.include_router(categorias.router)
app.include_router(productos.router)
app.include_router(ingredientes.router)

# Para correr: uvicorn app.main:app --reload

```



## PARTE 2 — El Frontend con React + TypeScript + TanStack Query

El frontend es la **interfaz que ve el usuario**. Construiremos una SPA (Single Page Application) con React y Vite. Usaremos **TypeScript** para tener tipado estático, **TanStack Query** para manejar el estado del servidor (datos que vienen de la API), y **Tailwind CSS** para los estilos.

### 2.1 — Crear el proyecto con Vite

Vite es una herramienta de construcción moderna, mucho más rápida que Create React App. Para iniciar el proyecto:

```
# En la carpeta raíz del repositorio
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install

# Instalar dependencias necesarias
npm install @tanstack/react-query react-router-dom axios
npm install -D tailwindcss @tailwindcss/vite
```

Para configurar Tailwind CSS v4, editá vite.config.ts:

```
// vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

Y en tu archivo index.css (o main.css) agregá al principio:

```
/* src/index.css */
@import 'tailwindcss';
```

### 2.2 — Estructura de carpetas del frontend

```
frontend/src/
├── main.tsx           # Punto de entrada React
├── App.tsx            # Configuración de rutas
├── api/              # Funciones que llaman a la API
│   ├── ingredientes.ts
│   ├── categorias.ts
│   └── productos.ts
├── types/            # Interfaces TypeScript
│   ├── ingrediente.ts
│   ├── categoria.ts
│   └── producto.ts
├── pages/            # Una página por módulo
│   ├── IngredientesPage.tsx
│   ├── CategoriasPage.tsx
│   └── ProductosPage.tsx
├── components/       # Componentes reutilizables
└── IngredienteModal.tsx
```

```
|   |─ CategoriaModal.tsx
|   |─ DataTable.tsx
|   └─ hooks/                # Custom hooks (opcional pero valorado)
```

## 2.3 — Definir los tipos (types/)

Los tipos en TypeScript son como **contratos**: definen exactamente qué forma tiene un objeto. Deben coincidir con los schemas que devuelve tu API. Para cada entidad creá un archivo en types/:

```
// src/types/ingrediente.ts

// Lo que devuelve la API (corresponde a IngredienteRead del backend)
export interface Ingrediente {
  id: number;
  nombre: string;
  descripcion: string | null;
  es_alergeno: boolean;
}

// Lo que mandamos al crear (corresponde a IngredienteCreate del backend)
export interface IngredienteCreate {
  nombre: string;
  descripcion?: string;
  es_alergeno: boolean;
}

// Lo que mandamos al editar (todo opcional)
export interface IngredienteUpdate {
  nombre?: string;
  descripcion?: string;
  es_alergeno?: boolean;
}
```

 Mantener la paridad entre los schemas Python y las interfaces TypeScript es la clave de una integración limpia. Cuando cambies algo en el backend, acordate de actualizar el tipo correspondiente en el frontend.

## 2.4 — Capa de llamadas a la API (api/)

Centralizá todas las llamadas HTTP en la carpeta `api/`. De esta forma, si la URL base de tu API cambia, solo tenés que tocar un archivo. Usaremos axios o fetch nativo:

```
// src/api/ingredientes.ts
import { Ingrediente, IngredienteCreate, IngredienteUpdate } from
'../types/ingrediente'

const BASE_URL = 'http://localhost:8000'

export async function getIngredientes(): Promise<Ingrediente[]> {
  const res = await fetch(`${BASE_URL}/ingredientes`)
  if (!res.ok) throw new Error('Error al obtener ingredientes')
  return res.json()
}
```

```

export async function createIngrediente(data: IngredienteCreate):
Promise<Ingrediente> {
  const res = await fetch(`${BASE_URL}/ingredientes`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  })
  if (!res.ok) throw new Error('Error al crear ingrediente')
  return res.json()
}

export async function updateIngrediente(id: number, data: IngredienteUpdate):
Promise<Ingrediente> {
  const res = await fetch(`${BASE_URL}/ingredientes/${id}`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  })
  if (!res.ok) throw new Error('Error al actualizar ingrediente')
  return res.json()
}

export async function deleteIngrediente(id: number): Promise<void> {
  const res = await fetch(`${BASE_URL}/ingredientes/${id}`, { method: 'DELETE' })
  if (!res.ok) throw new Error('Error al eliminar ingrediente')
}

```

## 2.5 — TanStack Query: useQuery y useMutation

TanStack Query resuelve el problema más difícil del frontend: **sincronizar el estado del servidor con la interfaz**. Sin él, tendrías que manejar manualmente los estados de loading, error, y la re-carga de datos después de una mutación. Configúralo en main.tsx:

```

// src/main.tsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { BrowserRouter } from 'react-router-dom'
import App from './App'
import './index.css'

const queryClient = new QueryClient()

ReactDOM.createRoot(document.getElementById('root')!).render(
  <QueryClientProvider client={queryClient}>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </QueryClientProvider>
)

```

Ahora veamos cómo usar useQuery y useMutation en una página real:

```

// src/pages/IngredientesPage.tsx
import { useState } from 'react'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

```

```

import { getIngredientes, deleteIngrediente } from '../api/ingredientes'
import { Ingrediente } from '../types/ingrediente'
import IngredienteModal from '../components/IngredienteModal'

export default function IngredientesPage() {
  const queryClient = useQueryClient()
  const [modalOpen, setModalOpen] = useState(false)
  const [editando, setEditando] = useState<Ingrediente | null>(null)

  // useQuery: gestiona la carga de datos del servidor
  // 'ingredientes' es la query key: TanStack Query la usa como cache key
  const { data: ingredientes, isLoading, isError } = useQuery({
    queryKey: ['ingredientes'],
    queryFn: getIngredientes,
  })

  // useMutation: gestiona operaciones que modifican datos
  const deleteMutation = useMutation({
    mutationFn: deleteIngrediente,
    onSuccess: () => {
      // Invalida el cache de 'ingredientes' → fuerza un re-fetch
      queryClient.invalidateQueries({ queryKey: ['ingredientes'] })
    },
  })

  if (isLoading) return <div className='p-4 text-center'>Cargando...</div>
  if (isError) return <div className='p-4 text-red-500'>Error al cargar
  datos</div>

  return (
    <div className='p-6'>
      <div className='flex justify-between items-center mb-4'>
        <h1 className='text-2xl font-bold text-gray-800'>Ingredientes</h1>
        <button
          onClick={() => { setEditando(null); setModalOpen(true) }}
          className='bg-blue-600 text-white px-4 py-2 rounded-lg
hover:bg-blue-700'
        >
          + Nuevo Ingrediente
        </button>
      </div>

      <table className='w-full border-collapse bg-white shadow rounded-lg'>
        <thead className='bg-gray-50'>
          <tr>
            <th className='text-left p-3 text-gray-600'>Nombre</th>
            <th className='text-left p-3 text-gray-600'>Alérgeno</th>
            <th className='p-3'>Acciones</th>
          </tr>
        </thead>
        <tbody>
          {ingredientes?.map(ing => (
            <tr key={ing.id} className='border-t hover:bg-gray-50'>
              <td className='p-3'>{ing.nombre}</td>
              <td className='p-3'>
                {ing.es_alergeno
                  ? <span className='bg-red-100 text-red-700 px-2
py-1 rounded text-sm'>Si</span>

```

```

: <span className='bg-green-100 text-green-700
px-2 py-1 rounded text-sm'>No</span>
    }
  </td>
  <td className='p-3 flex gap-2 justify-center'>
    <button
      onClick={() => { setEditando(ing);
setModalOpen(true) }}
      className='text-blue-600 hover:underline
text-sm'
    >Editar</button>
    <button
      onClick={() => deleteMutation.mutate(ing.id)}
      className='text-red-600 hover:underline
text-sm'
    >Eliminar</button>
  </td>
</tr>
  )}}
</tbody>
</table>

  {modalOpen && (
    <IngredienteModal
      ingrediente={editando}
      onClose={() => setModalOpen(false)}
    />
  )}
</div>
)
}

```

## 2.6 — El Modal con formulario (create + edit)

El modal sirve tanto para crear como para editar. La clave está en que si recibe un prop `ingrediente` poblado, está en modo edición; si recibe `null`, está en modo creación. Internamente usa **useMutation** para crear o actualizar, e **invalidateQueries** para refrescar la lista al cerrar:

```

// src/components/IngredienteModal.tsx
import { useState } from 'react'
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { createIngrediente, updateIngrediente } from '../api/ingredientes'
import { Ingrediente, IngredienteCreate } from '../types/ingrediente'

interface Props {
  ingrediente: Ingrediente | null // null = crear, objeto = editar
  onClose: () => void
}

export default function IngredienteModal({ ingrediente, onClose }: Props) {
  const queryClient = useQueryClient()
  const esEdicion = ingrediente !== null

  // Inicializamos el formulario con los datos existentes si editamos
  const [form, setForm] = useState<IngredienteCreate>({
    nombre: ingrediente?.nombre ?? '',
    descripcion: ingrediente?.descripcion ?? '',
  })
}

```

```

    es_alergeno: ingrediente?.es_alergeno ?? false,
  })

  const mutation = useMutation({
    mutationFn: esEdicion
      ? (data: IngredienteCreate) => updateIngrediente(ingrediente!.id, data)
      : createIngrediente,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['ingredientes'] })
      onClose()
    },
  })

  return (
    // Overlay oscuro de fondo
    <div className='fixed inset-0 bg-black/50 flex items-center justify-center z-50'>
      <div className='bg-white rounded-xl shadow-xl p-6 w-full max-w-md'>
        <h2 className='text-xl font-bold mb-4'>
          {esEdicion ? 'Editar Ingrediente' : 'Nuevo Ingrediente'}
        </h2>

        <div className='space-y-4'>
          <div>
            <label className='block text-sm font-medium text-gray-700 mb-1'>Nombre</label>
            <input
              type='text'
              value={form.nombre}
              onChange={e => setForm({ ...form, nombre:
e.target.value })}
              className='w-full border rounded-lg px-3 py-2
focus:outline-none focus:ring-2 focus:ring-blue-500'
            />
          </div>
          <div className='flex items-center gap-2'>
            <input
              type='checkbox'
              id='alergeno'
              checked={form.es_alergeno}
              onChange={e => setForm({ ...form, es_alergeno:
e.target.checked })}
            />
            <label htmlFor='alergeno' className='text-sm
text-gray-700'>Es alérgeno</label>
          </div>
        </div>

        {mutation.isError && (
          <p className='text-red-500 text-sm mt-2'>Ocurrió un error.
Revisá los datos.</p>
        )}

        <div className='flex justify-end gap-3 mt-6'>
          <button onClick={onClose}
            className='px-4 py-2 text-gray-600 hover:text-gray-900'>
            Cancelar
          </button>
          <button
            onClick={() => mutation.mutate(form)}

```

```

        disabled={mutation.isPending}
        className='bg-blue-600 text-white px-4 py-2 rounded-lg
hover:bg-blue-700 disabled:opacity-50'
      >
        {mutation.isPending ? 'Guardando...' : 'Guardar'}
      </button>
    </div>
  </div>
</div>
)
}

```

## 2.7 — Navegación con react-router-dom

React Router convierte tu app en una SPA con múltiples páginas. Configuralo en `App.tsx`. El parcial exige al menos una **ruta dinámica** (con parámetro en la URL, como `/productos/:id`):

```

// src/App.tsx
import { Routes, Route, NavLink } from 'react-router-dom'
import IngredientesPage from './pages/IngredientesPage'
import CategoriasPage from './pages/CategoriasPage'
import ProductosPage from './pages/ProductosPage'
import ProductoDetallePage from './pages/ProductoDetallePage'

export default function App() {
  return (
    <div className='min-h-screen bg-gray-100'>
      <nav className='bg-white shadow-sm px-6 py-4 flex gap-6'>
        <NavLink to='/ingredientes'
          className={({ isActive }) =>
            isActive ? 'text-blue-600 font-semibold' : 'text-gray-600
hover:text-blue-600'
          >Ingredientes</NavLink>
        <NavLink to='/categorias'
          className={({ isActive }) =>
            isActive ? 'text-blue-600 font-semibold' : 'text-gray-600
hover:text-blue-600'
          >Categorías</NavLink>
        <NavLink to='/productos'
          className={({ isActive }) =>
            isActive ? 'text-blue-600 font-semibold' : 'text-gray-600
hover:text-blue-600'
          >Productos</NavLink>
      </nav>

      <main className='max-w-5xl mx-auto py-8 px-4'>
        <Routes>
          <Route path='/ingredientes' element={<IngredientesPage />} />
          <Route path='/categorias' element={<CategoriasPage />} />
          <Route path='/productos' element={<ProductosPage />} />
          /* Ruta dinámica: el :id se captura con useParams */
          <Route path='/productos/:id' element={<ProductoDetallePage />} />
        </Routes>
      </main>
    </div>
  )
}

```

```

        </main>
      </div>
    )
  }

```

Para leer el parámetro :id en ProductoDetallePage:

```

// src/pages/ProductoDetallePage.tsx
import { useParams } from 'react-router-dom'
import { useQuery } from '@tanstack/react-query'
import { getProducto } from '../api/productos'

export default function ProductoDetallePage() {
  // useParams lee los parámetros dinámicos de la URL actual
  const { id } = useParams<{ id: string }>()

  const { data: producto, isLoading } = useQuery({
    queryKey: ['productos', id], // incluir el id en la key evita cache
    queryFn: () => getProducto(Number(id)),
    enabled: !!id, // solo ejecuta la query si id tiene valor
  })

  if (isLoading) return <div>Cargando...</div>

  return (
    <div>
      <h1>{producto?.nombre}</h1>
      <p>Precio: ${producto?.precio_base}</p>
      {/* Mostrar categorías e ingredientes relacionados */}
      <h2>Ingredientes</h2>
      {producto?.ingrediente_links?.map(link => (
        <div key={link.ingrediente_id}>
          {link.ingrediente?.nombre} - {link.cantidad}
          {link.unidad_medida_id}
        </div>
      ))}
    </div>
  )
}

```

## PARTE 3 — Integración, Tips para el Video y Checklist

### 3.1 — Flujo de una operación completa

Entender el flujo completo de una operación (por ejemplo, crear un ingrediente) te va a ayudar a explicar mejor en el video:

1. Usuario llena el formulario y hace click en 'Guardar'
- ↓
2. IngredienteModal llama a mutation.mutate(form)
- ↓



```
3. TanStack Query ejecuta createIngrediente(form)
  ↓
4. fetch hace POST /ingredientes con el JSON del form
  ↓
5. FastAPI recibe la request → valida con Pydantic
  ↓
6. El router delega a ingrediente_service.create()
  ↓
7. SQLAlchemy hace INSERT INTO ingrediente (...) VALUES (...)
  ↓
8. PostgreSQL guarda el registro y devuelve el id generado
  ↓
9. session.refresh() carga el id de vuelta en el objeto
  ↓
10. FastAPI serializa con IngredienteRead y responde 201
  ↓
11. onSuccess: invalidateQueries(['ingredientes'])
  ↓
12. TanStack Query re-fetcha la lista → la UI se actualiza
```

## 3.2 — Errores comunes y cómo evitarlos

Estos son los problemas que más frecuentemente aparecen en proyectos como este:

- **CORS error en el browser:** Aparece cuando FastAPI no tiene configurado el middleware de CORS. Verificá que el puerto de Vite (generalmente 5173) esté en `allow_origins`.
- **'Table does not exist':** Ocurre cuando los modelos no se importaron antes del `create_all()`. Importá todos los modelos en `main.py` antes de llamar a `create_db_and_tables()`.
- **'n+1 queries' con las relaciones:** Si hacés `session.get(Producto)` y después accedés a `producto.ingrediente_links`, SQLAlchemy hace una query por cada producto. Usá `selectinload` o `joinedload` de SQLAlchemy si listás muchos productos con sus relaciones.
- **Estado del formulario no se resetea:** Si abrís el modal de creación después de haber editado, el `useState` puede conservar los valores anteriores. Resóvelo usando la prop `ingrediente` como `dependency` del estado inicial, o reseteando manualmente en `onClose`.
- **TypeScript 'any' en todas partes:** Define bien tus interfaces. Usar `'any'` es la señal más clara de que el tipado no se está aprovechando. El evaluador lo nota.

## 3.3 — Preparación del video (consejos docentes)

El video representa el 20% de la nota según la rúbrica. Estos consejos te ayudan a aprovecharlo al máximo:

1. **Estructurá el guión antes de grabar:** Sabé exactamente qué vas a mostrar en cada minuto. Un video improvisado se nota y consume el tiempo rápidamente.
2. **Abrí los archivos clave antes de empezar:** Tené abiertos en pestañas separadas: el modelo de Producto, el router de ingredientes, la página `IngredientesPage.tsx`, y `pgAdmin` con las tablas.
3. **Mostrá primero que funciona, después explicá el código:** Hacé el CRUD completo en vivo (crear, editar, eliminar) para establecer que la app funciona. Después mostrá el código que lo hace posible.

4. **Nombrá los patrones técnicos:** Cuando muestres `back_populates`, decí explícitamente 'esta es la relación bidireccional'. Cuando hagas `invalidateQueries`, decí 'esto invalida el cache y fuerza un re-fetch'. Los evaluadores buscan que conozcas la terminología.
5. **Mostrá un error de validación:** Intentá guardar un ingrediente con el nombre vacío o con precio negativo. Que se vea el mensaje de error de Pydantic en la consola o en la UI.
6. **Usá zoom en el código:** El código a pantalla completa con font size 16-18px es cómodo para leer en video. Usá `Ctrl+=` para agrandar.

### 3.4 — README.md recomendado

El README es lo primero que ve el evaluador al entrar al repositorio. Debe incluir:

```
# Nombre del Proyecto

Descripción breve del sistema.

## Video de presentación
[Ver video en YouTube] (URL_DEL_VIDEO)

## Tecnologías
- Backend: FastAPI, SQLAlchemy, PostgreSQL
- Frontend: React 18, TypeScript, TanStack Query, Tailwind CSS 4

## Cómo ejecutar
### Backend
```bash
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# Crear archivo .env con DATABASE_URL
uvicorn app.main:app --reload
```

### Frontend
```bash
cd frontend
npm install
npm run dev
```
```

---

### Reflexión final del docente

Este parcial no es sobre memorizar APIs o frameworks. Es sobre demostrar que entendés **por qué** las cosas se hacen de cierta manera: por qué separamos `models` de `schemas`, por qué usamos `invalidateQueries` en lugar de recargar la página, por qué la sesión de `SQLModel` usa `yield`. Si podés explicar el porqué mientras mostrás el código, el evaluador va a ver que realmente aprendiste.

No te frustres si algo no funciona al 100%. La rúbrica valora la honestidad técnica: explicar qué intentaste y por qué falló demuestra más comprensión que una app perfecta que no podés justificar.

**¡Éxitos!** — Esta guía fue diseñada para que seas vos quien construya y entienda, no para que copies.